Indicator Control Module Documentation

Name: LABIB SERJ
Assignment: Embedded Software Development Internship Assignment
Company: Gray Mobility

---

1. Project Title

Indicator Control Module using STM32 and Simulink Generated Code

---

2. Abstract

This project implements an automotive-style Indicator Control ECU using an STM32 Nucleo-F446RE microcontroller. The system supports Left Indicator, Right Indicator, Indicator Switching, and Hazard Light functionality. The application logic is implemented in MATLAB Simulink and integrated into STM32 firmware using Embedded Coder generated code. The system uses a 100 ms scheduler, button debounce logic, PWM LED control, and UART logging for event monitoring.

---

3. Objective

The objective of this project is to design and implement embedded software for an Indicator Control Module that:

- Reads button inputs
- Controls LEDs using PWM
- Implements indicator state machine logic
- Implements hazard light logic
- Uses a 100 ms scheduler
- Logs system events through UART
- Demonstrates layered embedded software architecture

---

4. Hardware Platform

Microcontroller

- STM32 Nucleo F446RE

Pin Mapping

Function	Pin
Left LED PWM	PA0
Right LED PWM	PA1
UART TX	PA2
UART RX	PA3
Left Button	PB4

Right Button	PB5
Scheduler Timer	TIM6
PWM Timer	TIM2

---

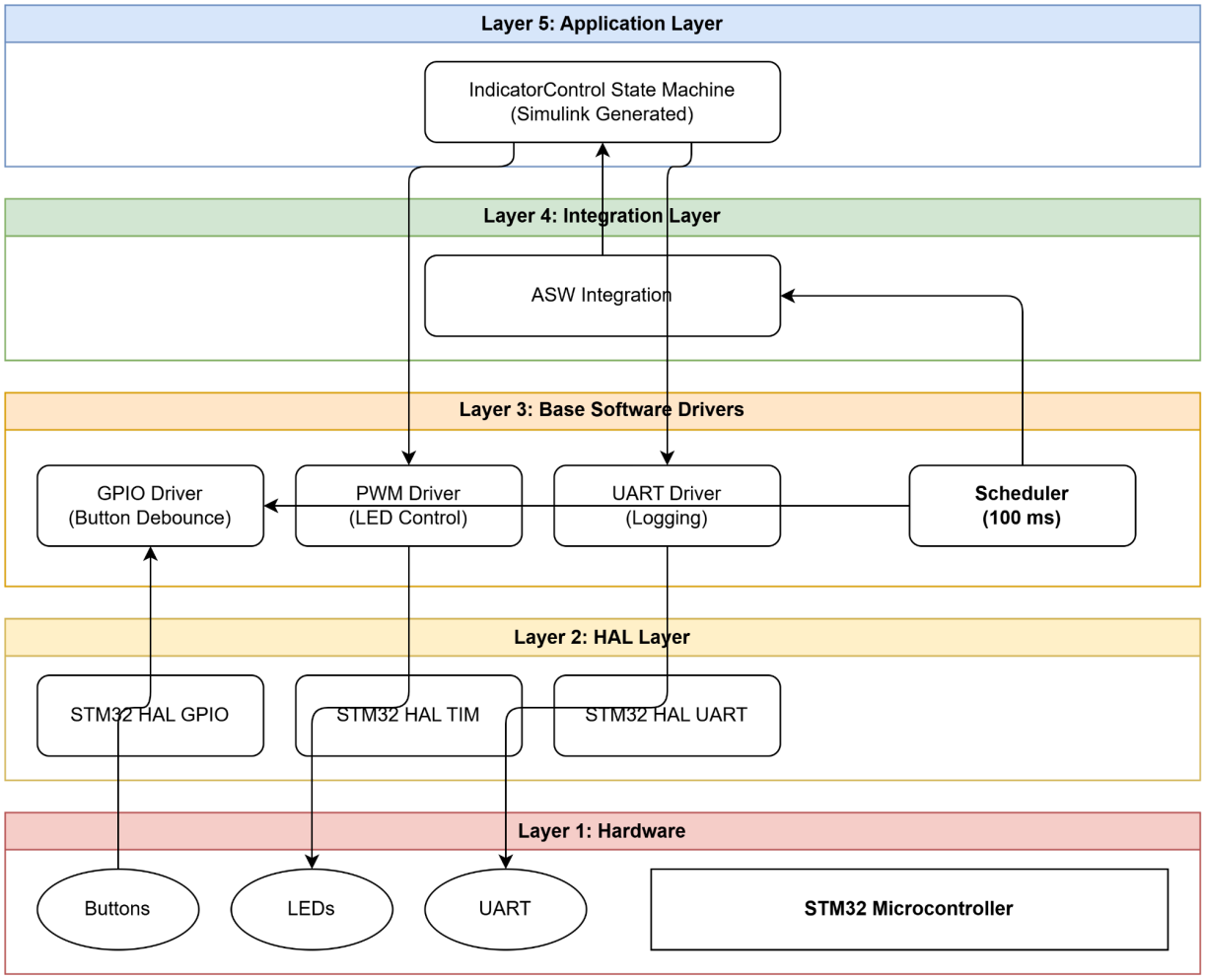
5. Software Architecture

The software is implemented using a layered architecture:

Software Layers

1. **Hardware Abstraction Layer (HAL)** – Generated by STM32CubeMX
2. **Base Software (BSW)**
    - GPIO Driver
    - PWM Driver
    - UART Driver
    - Scheduler
3. **ASW Integration Layer**
    - Integrates Simulink generated code with drivers
4. **Application Software (ASW)**
    - Indicator state machine generated from MATLAB Simulink

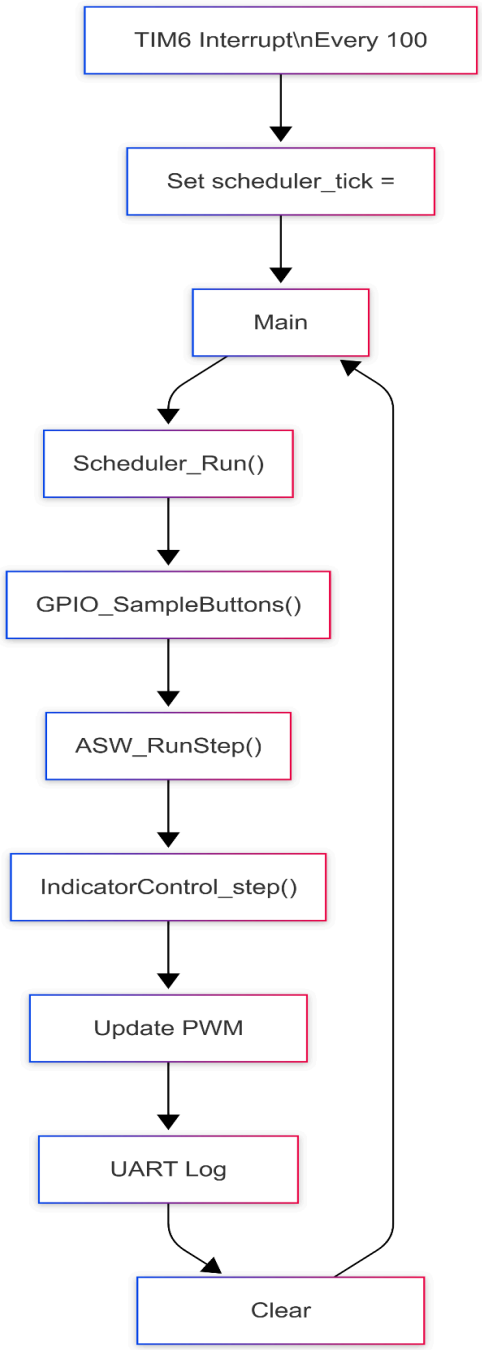
Architecture Diagram



6. Scheduler Design

A cooperative scheduler running at 100 ms is implemented using TIM6 interrupt.

Scheduler Flow



Timing Parameters

Function	Time
Scheduler Period	100 ms
Button Hold Time	1 sec
Blink Period	300 ms
PWM Frequency	1 kHz

---

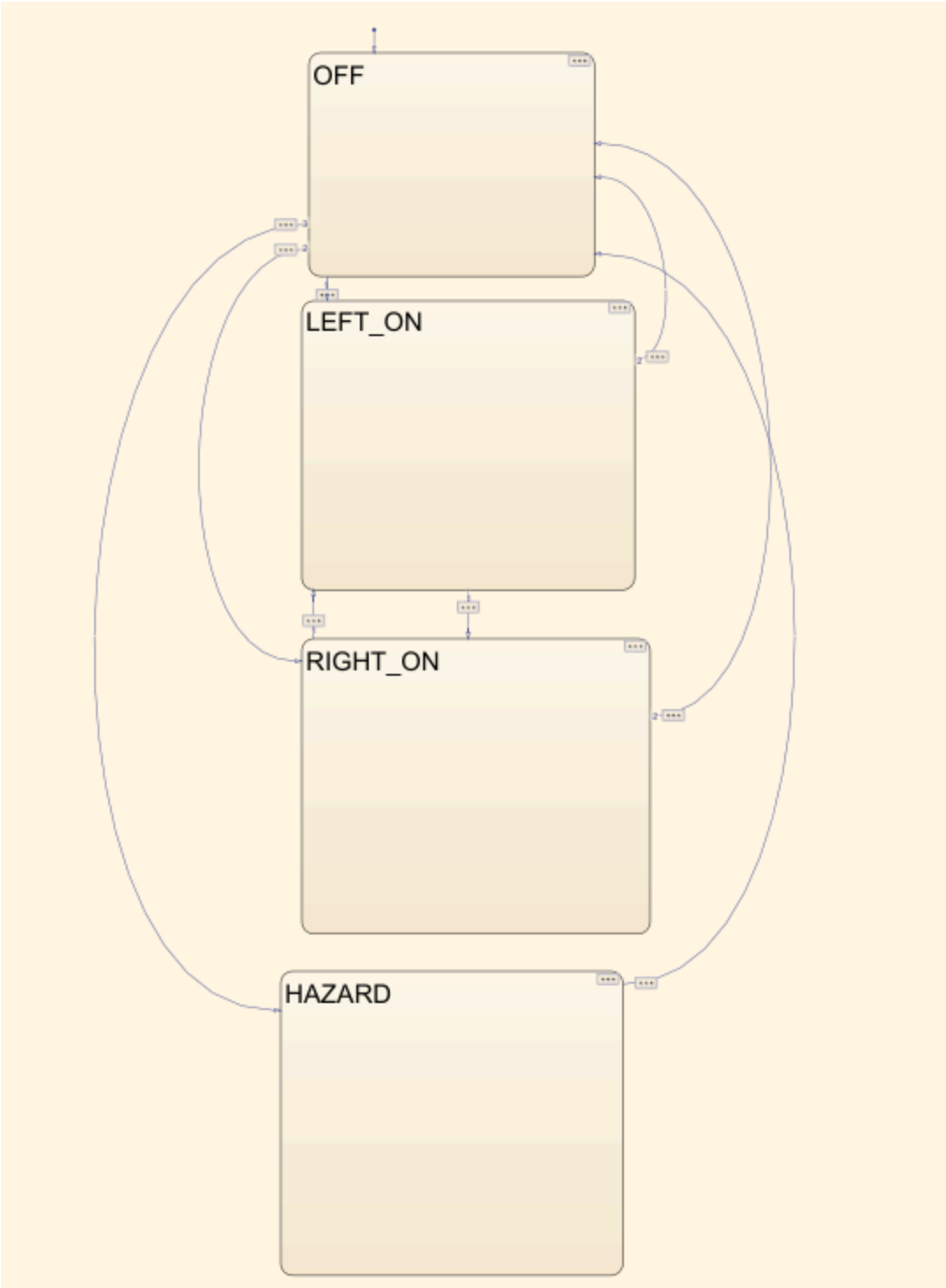
7. Indicator State Machine

States

- OFF
- LEFT_ON
- RIGHT_ON
- HAZARD

State Transitions

Condition	Transition
Left button held 1 sec	OFF → LEFT
Right button held 1 sec	OFF → RIGHT
Both buttons held 1 sec	OFF → HAZARD
Left held while RIGHT	RIGHT → LEFT
Right held while LEFT	LEFT → RIGHT
Press any button in HAZARD	HAZARD → OFF


OFF
LEFT_ON
RIGHT_ON
HAZARD

8. Button Debounce Logic

Buttons are sampled every 100 ms. A button state is accepted only after it remains stable for consecutive samples. A hold counter increments every 100 ms while the button is pressed. When the hold counter reaches 10 counts, it corresponds to a 1 second button hold.

---

9. UART Logging

The system logs events with timestamps.

Example UART Logs

[33277 ms] LEFT_INDICATOR_OFF

[33279 ms] LEFT_LED_TOGGLE

[33282 ms] RIGHT_INDICATOR_OFF

[33284 ms] RIGHT_LED_TOGGLE

[33377 ms] LEFT_BUTTON_PRESSED

[33477 ms] RIGHT_BUTTON_PRESSED

[33577 ms] LEFT_INDICATOR_ON

[33579 ms] LEFT_LED_TOGGLE

[33582 ms] RIGHT_INDICATOR_ON

[33584 ms] RIGHT_LED_TOGGLE

[33587 ms] HAZARD_ON

UART logging helps verify system timing, state transitions, and debugging.

---

10. Testing and Validation

Functional Tests

- Left Indicator ON/OFF
- Right Indicator ON/OFF
- Indicator Switching
- Hazard Mode Activation
- Hazard Mode Exit
- Blink Timing Verification

Robustness Tests

- Rapid Button Press Test
- LED Disconnect Test
- Power Reset Test

---

11. Project Folder Structure

```
Core/
  Inc/
    gpio_driver.h
    pwm_driver.h
    uart_driver.h
    scheduler.h
    asw_integration.h
    IndicatorControl.h
```

IndicatorControl_private.h

IndicatorControl_types.h

rt_defines.h

rtwtypes.h

Src/

gpio_driver.c

pwm_driver.c

uart_driver.c

scheduler.c

asw_integration.c

IndicatorControl.c

---

12. GitHub Repository

GitHub Link: _____

---

13. Demo Video

Demo Video Link (Google Drive):

https://drive.google.com/drive/folders/1PblGP-97_xRYWnrP7eygRF27OttxQrPz?usp=drive_link

---

14. Conclusion

This project demonstrates a complete embedded automotive-style indicator control module using STM32 microcontroller, cooperative scheduler, layered software architecture, PWM LED control, button debounce logic, UART logging, and Simulink generated state machine integration. The system follows modular embedded software design principles and demonstrates real-world embedded system development practices.

End of Document